

Aumazing Technical Defense Reviewer

A practical study guide for the technical portion of the capstone defense

Use the short answer first. The deeper notes give you the implementation detail to use when a panelist asks “how?” or “why?”.

Core story	An offline-first Flutter and Flame learning app records gameplay locally, classifies four educational areas, and turns the result into a sequential My Path.
Four areas	Communication • Social Interaction • Play Skills • Attention
Three bands	Needs Support • Emerging • Strength
Critical boundary	The system is an educational support prototype. Synthetic, rubric-derived labels do not establish clinical validity.

Keep these answers precise

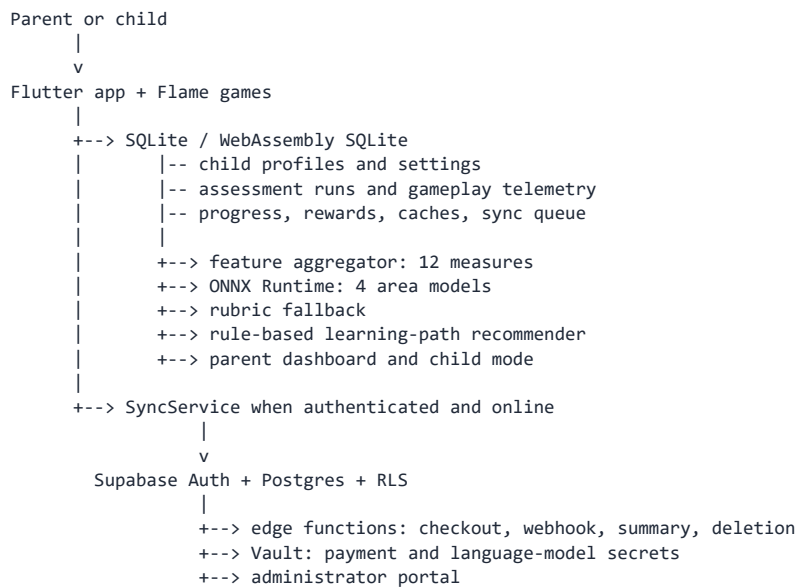
- Four local ONNX models classify the four areas; the optional Gemini 2.5 Flash service only turns minimized results into narrative text.
- The deterministic rubric is a fallback and a readable scoring path. It is not proof that the model is accurate.
- The current recommendation path is rule-based. The manuscript calls it hybrid, but a separate content-similarity component is not in the current code.
- 902 passing tests belong to one identified CI release run. They do not prove clinical effectiveness or universal device compatibility.

Source basis: supplied capstone PDF and repository implementation checked on 16 September 2026. Page references in the reviewer use the PDF page numbering where mentioned.

The answer to memorize first

Aumazing is an offline-first Flutter application for parent-guided learning activities. Flutter and Flame run the child-facing games, while SQLite stores profiles, gameplay telemetry, assessment runs, and progress on the device. After the four-game pre-assessment, the app aggregates twelve gameplay measures and classifies four educational skill areas - Communication, Social Interaction, Play Skills, and Attention - into Needs Support, Emerging, or Strength. Four bundled ONNX models run locally through ONNX Runtime; if local inference is unavailable, the app can use a cloud prediction service and then deterministic rubric scoring. Rule-based recommendation logic maps the resulting areas to a sequential My Path. Supabase provides authentication, synchronized family data, row-level security, and server-side functions for operations that must not be trusted to the client. Parents see dashboards, history, controls, reports, rewards, and the therapy directory; administrators use a separate Flutter web portal. The system is technically demonstrated, but the model is not a clinical diagnostic instrument: its training labels are synthetic and rubric-derived, and representative child and practitioner validation is still required.

The 30-second system map



External boundaries: PayMongo, Google identity, optional cloud assessment, optional Gemini 2.5 Flash summary, map tiles, device location, and device maps app.

The design choice to defend is local function with optional synchronization. Connectivity affects cloud durability, cross-device availability, and some external features; it does not stop the core assessment and learning loop after the first sign-in.

Component responsibilities

Component	Responsibility	Technical detail to mention
Flutter parent UI	Account, profile, dashboard, assessment setup, reports, settings	Feature-based screens under <code>apps/main_app/lib/features/</code>
Flame activity layer	Touch-based child activities and immediate feedback	Twelve registered games under <code>packages/game_core/lib/src/games/</code>
Shared UI/audio/haptic packages	Consistent controls, voice prompts, vibration, themes	Reused by the main app and game lab
GameRegistry	Single catalog of game metadata and factories	Holds names, skill categories, logos, and the four assessment IDs
LocalDbService	Device persistence	SQLite on Android; WebAssembly SQLite in the browser
SyncService	Upload and hydration	Foreign-key order, batched upserts, retry/backoff, soft deletes
Feature aggregator	Converts gameplay to model inputs	Twelve named features; Dart port mirrors Python aggregator
ONNX Runtime	Local classification	Four models: communication, social, play, attention
RubricScoringService	Deterministic fallback and readable bands	Uses cached/admin-configured thresholds
Recommendation rules	Selects modules and starting levels	Current implementation is rule-based
Supabase	Identity, authoritative cloud copy, RLS, reference data	Postgres 17, Singapore region, 26 public tables at audit snapshot
Edge functions	Trusted server operations	<code>create-checkout</code> , <code>paymongo-webhook</code> , <code>summarize-assessment</code> , <code>delete-account</code>
Admin portal	Controlled management and review	Dashboard, Beta Review, Accounts, Games, Centers, Rubric, Audit Log, Payments

End-to-end assessment trace

1. The parent selects a child and opens the pre-assessment.
2. The parent sees the educational-use disclaimer and sensory-consent choice. Research consent is optional and separate.
3. The child plays the four assessment games in this order: Copy Me, Do What I Say, My Turn, Your Turn, and Match It. “My Turn, Your Turn” is one game title.
4. Each round records telemetry locally. The current round plan gives Copy Me four rounds (music, haptic, baseline, and combined) and gives the other three assessment games three rounds each. Practice activities use three rounds.
5. The app stores score, item count, errors, response time, completion, retries, hints, prompt dependency, idle time, off-target/random touches, turn-taking measures, and sensory condition data where available.
6. The feature aggregator derives twelve numeric inputs:
 - overall accuracy;
 - average response time per item;
 - task-completion rate;
 - retries;
 - hints;
 - prompt-dependency score;
 - idle time;
 - invalid/random touches;
 - one accuracy value for each assessment game.
7. Four local ONNX models predict one level per area. The app records the source of the result.

8. If local inference is unavailable, AiPredictionFallbackService tries the cloud prediction path and then the prepared deterministic rubric path.
9. LocalRecommendationRules maps non-Strength areas to games and starting levels. LearningPathService orders the path by the most-supported area first and unlocks steps sequentially.
10. The parent sees Overall Performance, Game Results, the four-area Developmental Profile, Recommended Settings, and Recommended Activities.
11. The result is saved locally immediately. Synchronization runs independently when the account is authenticated and online.
12. After the path is complete, a later post-assessment can be compared only when the child, time order, completed levels, and shared areas match.

How to explain the classifier

What the model predicts

The current design is a multi-output three-class classifier with ordered educational bands. It produces four separate outputs, each encoded as:

Value	Display label
0	Needs Support
1	Emerging
2	Strength

The four areas are Communication, Social Interaction, Play Skills, and Attention. The labels have an intended order, but the training code uses ordinary multiclass XGBoost objectives; it is not a specialized ordinal-regression model. These are educational activity bands for choosing what to play next. They are not a diagnosis, severity score, or probability of having a condition.

Why XGBoost was selected

The inputs are structured tabular measures rather than images or raw audio: accuracy, latency, errors, prompts, idle time, and game-specific performance. XGBoost is suitable for this type of data, supports nonlinear interactions, works with small-to-medium structured datasets, and can expose feature importance. ONNX export allows the trained model to run in the mobile application without a network request.

How one feature is computed

The current Python and Dart aggregators use adjusted session accuracy:

```
adjusted_accuracy = score / (score + error_count)
```

This retains information that raw `score / total_items` can hide when a child eventually reaches the correct answer after failed attempts. Average response time is calculated per item, and per-game accuracy is averaged over sessions for the corresponding game. The overall task-completion feature is a proxy: it is the fraction of sessions with `score > 0`, not the fraction of all tasks completed perfectly. Missing per-game data use the configured default of 0.5, so a default is not evidence that a child demonstrated that skill.

The honest model-validation answer

The repository contains an 800-row synthetic training dataset and a training script with five-fold cross-validation. However, each training label is derived from the same engineered rubric that defines the features. High agreement on that dataset shows that the implementation can reproduce the designed mapping; it does not establish predictive validity for real children, practitioner agreement, or clinical usefulness. The manuscript correctly records held-out, child-separated evaluation and representative validation as remaining work. The live audit also recorded model results as rubric-based; model files being present is not evidence that an ONNX prediction was executed for those rows.

If asked for an exact accuracy number, say:

We do not present a headline accuracy number as real-world model validity. The data are synthetic and rubric-labelled, and the current live audit rows were recorded as rubric-based. The defense claim is an implemented assessment pipeline; a held-out, child-separated ONNX evaluation is still required before quoting model performance.

Keep three outputs separate

- Assessment evidence: gameplay telemetry and the twelve-number feature vector.
- Assessment result: either the four local/cloud model outputs or the deterministic rubric result, with its source recorded.
- Parent narrative: an optional Gemini 2.5 Flash summary of minimized bands, percentages, and recommendation names. It does not calculate the numerical scores, choose modules, or turn the result into a diagnosis.

How to explain the rubric fallback

The rubric is deterministic and remains available offline. It reads the same gameplay evidence and assigns readable bands by domain:

- Play Skills: Match It and Copy Me, using accuracy, completion, prompt dependency, and retries.
- Communication: Copy Me and Do What I Say, using accuracy and prompt dependency.
- Social Interaction: My Turn, Your Turn, using turn-taking success and interruptions.
- Attention: all valid sessions, using idle time, random touches, and completion.
- Sensory preference: the sensory-round analyzer compares music-only, haptic-only, combined, and baseline performance where those metrics are available.

The default runtime threshold examples are 0.50 for Emerging, 0.80 for Strength, prompt dependency at or below 0.20 for a Strength result, and attention cutoffs of 5 and 15 seconds for sustained versus variable attention. The administrator portal can update the threshold row, and the client caches the last successful values for offline scoring.

Threshold issue to prepare for

The current repository also contains a training-data generator with different label constants: 0.40 and 0.70 for accuracy bands and marker-count rules for Attention. Do not claim that training labels and runtime fallback thresholds are already one identical validated rubric. If asked, say:

The implementation exposes both the training-label rubric and the runtime scoring rubric, and they must be reconciled before the model is treated as a validated measurement. Our current evidence supports the architecture and the fallback behavior; it does not support a claim that these thresholds have been normed on a real-child dataset.

This is a technical reconciliation item to close before submission if the panel expects one canonical threshold table.

How to explain the recommender

The current implementation is rule-based:

1. Read the four area levels.
2. Skip areas already at Strength.
3. For each remaining area, select the registered games mapped to that area.
4. Assign a starting level from the area level.
5. Deduplicate games, keeping the lowest required starting level.
6. Refine a game's starting level when its per-game adjusted accuracy is stronger.
7. Sort the resulting path so the greatest support need is addressed first.
8. Unlock the path sequentially as games are completed.

The manuscript uses the phrase “hybrid rule-based and content-based recommender,” but the current code implements rule-based mapping through `LocalRecommendationRules` and `LearningPathService`; a separate content-based similarity component is not present. If challenged, acknowledge the implementation accurately rather than defending a component that is not in the code.

Offline-first and synchronization

Why local-first was chosen

The target setting includes intermittent connectivity and metered data. A network-first architecture could interrupt a child's game or lose an assessment at the moment it is completed. The device therefore writes locally first, and the cloud acts as the synchronized authoritative copy for account-backed data.

Upload sequence

SyncService uploads in foreign-key order so parents exist before children:

```
children
-> assessment_runs
-> game_sessions
-> game_rounds
-> session_events
-> caregiver_questionnaires
-> assessment_results
-> module_recommendations
-> assessment_comparisons
```

Uploads use client-generated UUIDs and idempotent upserts. Batches are capped at 200 rows. A rejected row stays pending and retries after 30 seconds, 1 minute, 5 minutes, and 15 minutes. A fresh install hydrates the signed-in account's cloud rows into the local database without overwriting local state.

Conflict policy

The current policy is write order/latest upsert wins. There is no field-level merge or vector clock. This is acceptable for the current one-caregiver editing assumption and append-only gameplay rows, but it is a limitation for future multi-caregiver editing.

What is not fully synchronized at the audit snapshot

The manuscript records that the star ledger, costume unlocks, and some sensory tables remain local because matching live cloud schema was not fully applied. A reinstall can therefore lose those local-only records. Do not promise that every feature has cross-device recovery.

Security and privacy answers

Why client checks are not enough

The client is under the user's control. A modified app could bypass a Dart-only check, so authorization is enforced in Supabase policies and server functions. Client-side checks improve usability; database policies provide the security boundary.

Row-level security

At the audit snapshot, all 26 public tables had RLS enabled. Family rows are scoped through the caregiver's authenticated account identifier. Child-keyed rows reach the same ownership predicate through the child relationship. The live two-account probe reported zero rows for cross-family reads, zero affected rows for unauthorized updates/deletes, and refusal of an insert referencing another family's child.

Parent lock

Leaving Child Mode requires a parent gate. The PIN is salted and hashed with PBKDF2-HMAC-SHA256 and is not stored in recoverable form. Five failed attempts trigger a 60-second lockout. A word-code mode is available, and a parent-selected PIN can be used when appropriate.

Payment integrity

The client requests checkout creation from an edge function. The payment provider calls the webhook. The webhook verifies the HMAC signature, deduplicates provider events, and writes the entitlement using server privileges. The client cannot insert or update entitlement or payment rows. Payment and language-model secrets are held in the server vault.

Location privacy

Therapy-center coordinates are reference data. The parent's position is requested only when Find near me is tapped, held in memory, used in a Haversine distance calculation, and discarded. The app hands directions to the device maps application; it does not store a route.

Security gaps to state clearly

The manuscript records the following open items at the documentation snapshot:

- Premium expiry is not enforced by the client; it reads `is_premium` but not `expires_at`.
- The deployed webhook version lags the current repository fix.
- A personal access token remains retrievable in public repository history; revocation was unverified.
- Leaked-password protection is disabled.
- Five of fifteen database functions lack an explicit `search_path` setting.
- The migration ledger and live schema have drifted; the star-shop and child-music migrations were not fully applied.
- Development, evaluation, and demonstration share one Supabase project.
- Backup and point-in-time recovery settings were not verified.

These are not reasons to hide the design. They are the limits of the current technical evidence and the remediation list before production use.

Administrator and external integrations

Administrator portal

The administrator portal is a separate Flutter web app. Its navigation sections are:

1. Dashboard
2. Beta Review
3. Accounts
4. Games
5. Centers
6. Rubric
7. Audit Log
8. Payments

Admin status is based on membership in `admin_users`. The `is_admin()` check and sensitive operations run server-side. The portal renders the management UI; it is not the authorization boundary.

External services

Service	What crosses the boundary	Trust boundary
Supabase Auth	Email/password, Google identity, anonymous guest session, JWT	Identity and session issuance
Supabase Postgres	RLS-scoped family/reference rows	Database authorization
PayMongo	Checkout creation and signed webhook events	Server verifies signature
Gemini 2.5 Flash summary	Minimized skill bands, percentages, and recommendation names	Edge function; no child identifier; it does not assign the numerical scores or decide the recommended modules
Cloud assessment fallback	Feature vector and prediction	Optional second prediction tier; deployment status must be verified
OpenStreetMap tiles	Raster map tiles	Reference visualization
Device location	One position fix on locator use	Held in memory only
Device maps app	Center coordinates through geo/web handoff	Route is external to Aumazing
Google identity	OAuth token	Client identifier is build configuration

Testing and evidence

Use evidence categories precisely:

Evidence	What it supports	What it does not prove
902 passing Flutter tests in CI run 34164867343	Strong internal verification of covered code paths for the identified release revision	Every later commit, every device, or user acceptance
40 Deno payment-outcome tests	Signature/event outcome logic	Live-mode payment operation
OPPO CPH2711, Android 16/API 36	One physical Android compatibility result	Minimum API 24, low-end phones, tablets, or all Android versions
Desktop Chrome web audit	Browser demonstration behavior	Keyboard success; the audit found application controls were not reachable by keyboard
Remote UAT on Honor Pad 8X	One respondent's task and usability report; twelve games passed, ease 5/5, child-friendliness 4/5	Representative target-age ASD validation
Practitioner reviews	Feedback that led to revisions, such as the Copy Me hold	Completed signed practitioner acceptance; the final form was pending
Synthetic model data and CV script	Reproducible training pipeline and designed mapping	Real-child predictive validity or clinical accuracy
RLS and negative probes	Live cross-family isolation and admin refusal behavior	A complete security certification

A strong testing answer

We distinguish implementation, internal automated verification, integration evidence, and external validation. The release CI run gives us 902 passing Flutter tests and a clean analysis/build result for its identified revision. We also have live RLS and payment-boundary probes. The remaining evidence is release-specific retesting after later fixes, a complete minimum-device matrix, held-out child-separated model evaluation, and signed parent/practitioner validation. We therefore claim technical implementation and internal verification, not clinical effectiveness or universal compatibility.

Likely reviewer questions and model answers

Architecture

1. Why did you choose Flutter?

Flutter lets the team share one feature-based codebase across Android and web while keeping a consistent child-friendly interface. Shared packages separate UI, audio, haptics, and game logic, reducing duplicated behavior.

2. Why use Flame instead of ordinary Flutter widgets for the games?

The games require a 2D scene, touch interaction, movement, timing, and immediate feedback. Flame provides the activity/game loop while Flutter handles the surrounding screens, forms, navigation, and parent reporting.

3. What is the most important architectural decision?

Local-first persistence. The child can complete the assessment and practice loop without connectivity after first sign-in, and synchronization becomes a durability and cross-device concern.

4. What is the system of record?

For active interaction, the local SQLite store is the immediate system of record. For an authenticated account after synchronization, Supabase holds the authoritative cloud copy that a fresh install can rehydrate. The two copies are joined by client-generated UUIDs and synchronization metadata.

5. What happens when the app is killed in the middle of an assessment?

The run and gameplay state are persisted locally. The assessment can offer resumption within the configured seven-day window or restart. This is covered by the assessment state and persistence tests.

Assessment and AI

6. Why are there four assessment games?

They provide distinct evidence for imitation, verbal instruction, turn-taking, and matching/play. Their telemetry also maps cleanly to the four educational areas used by the result and recommendation views.

7. Why does one pre-assessment game have four rounds while the other three have three?

Copy Me runs the full four-round sensory evidence cycle: music, haptic, baseline, and combined. Do What I Say, My Turn, Your Turn, and Match It run three rounds because their combined round is not needed for the current sensory label and dropping it shortens the shared assessment. Practice uses three rounds. This is controlled by a shared round-count policy.

8. Why twelve features?

Eight are overall behavioral measures and four are per-game accuracy measures. Together they capture correctness, latency, completion, support dependence, attention markers, and domain-specific performance without sending raw media.

9. Why does the model run locally?

The assessment must work during connectivity interruption, and local inference reduces the need to send detailed behavioral telemetry to a server for every result. The trade-off is model asset size and the need to keep the Dart and Python feature order equivalent.

10. What is the fallback if ONNX cannot load?

The browser and native builds both have a local ONNX path: native builds use the platform runtime, while the browser uses ONNX Runtime Web through its WebAssembly bridge. If that local path is unavailable, the prediction service can try the cloud prediction path and then the deterministic rubric path. The result source is stored so the parent-facing outcome is not presented as if every tier were identical.

11. Is the classifier clinically validated?

No. It is an educational skill-band classifier used to select activities. The current model is trained on synthetic rubric-labelled data, so the evidence demonstrates the engineering pipeline and reproducibility, not clinical validity.

12. Why use synthetic data?

It allowed the team to test the end-to-end feature, training, export, and inference path before an ethically approved real-child dataset existed. The limitation is label circularity: labels come from the same rubric that defines the features.

13. What would a proper next evaluation look like?

Collect an ethically approved real-child dataset with qualified oversight, separate training and evaluation by child, report per-domain and per-band metrics, include confidence calibration, and compare outputs with qualified practitioner judgements. Keep children from the same household or participant out of both train and test partitions where leakage is

possible.

14. Why not use a deep neural network?

The input is structured tabular telemetry and the available dataset is small. XGBoost is a better fit for that data shape and supports a lightweight ONNX deployment. A deep model would add complexity without evidence that it improves this task.

15. How do you interpret confidence?

Confidence describes the model's certainty in the educational band prediction. It is not the likelihood of a diagnosis and should not be read as a clinical risk score.

16. Are your training labels and runtime thresholds the same?

The current repository shows a discrepancy: the runtime rubric defaults and the training-data generator use different accuracy constants. We should reconcile them before claiming one canonical validated rubric. At present, this is an implementation and validation limitation.

Recommendation and learning path

17. Is the recommender content-based, collaborative, or rule-based?

The current code is rule-based. It maps per-area levels and game metadata to modules and starting levels, then sequences and unlocks the path. The manuscript's content-based wording is broader than the implementation; no collaborative filtering or separate content-similarity model is present.

18. How is the next activity selected?

Non-Strength areas produce candidate games, duplicates are merged using the lowest starting level, per-game adjusted accuracy can raise a level, and the path is ordered by support need. Completed steps unlock the next step sequentially.

19. What happens if an administrator disables a game?

The active-game set filters the path. The recommendation service skips unavailable games, so the path is built only from currently active registered content.

20. What prevents a child from entering parent screens?

Child mode has a parent gate. Leaving it requires the configured word code or parent PIN, with salted hashing and failed-attempt throttling. The gate is a usability and access barrier; database authorization separately protects cloud data.

Data and synchronization

21. How do you prevent duplicate rows after retry?

Rows receive UUIDs on the device, and uploads use upsert by row identifier. Retrying the same record therefore remains idempotent rather than inserting another copy.

22. Why upload in foreign-key order?

The parent row must exist before a dependent child or session row can satisfy its foreign key. Ordered uploads reduce avoidable constraint failures during a batch.

23. What if one row in a batch fails?

Per-record failure handling leaves that row pending and allows other records to complete. The pending state is visible to the app, and retry uses backoff.

24. How are conflicts resolved?

Latest write order wins for the same UUID. There is no field-level merge. This is acceptable under the current single-caregiver editing assumption but should be revisited for multi-caregiver use.

25. What data is intentionally not collected?

The system does not store face images, photographs, voice recordings, voice samples, parent GPS coordinates, diagnosis documents, chat content, or advertising identifiers. It does store a child's display name and exact birth date for profile and age-based behavior; that collection is disclosed.

26. Does the therapy locator store a parent's location?

No. It requests one position fix on demand, ranks cached center data using Haversine distance, and discards the position when the screen operation ends.

Security and backend

27. Why is the Supabase publishable key safe to ship?

It is a client key by design. It does not grant service-role privileges; RLS policies scope table access. The service-role key and external secrets stay inside server-side functions and Vault.

28. Can a client grant itself Premium?

No. The client can read its own entitlement, but the table does not grant authenticated clients write access. The webhook is the only writer after signature verification and checkout binding.

29. Why does the webhook not require a JWT?

The payment provider is the caller. It authenticates through its HMAC signature, which the webhook verifies against the raw request body and the correct test/live signature field.

30. How are administrator operations protected?

The portal checks membership for UI feedback, but the security boundary is server-side. `is_admin()` and sensitive SECURITY DEFINER functions reject non-admin sessions and write audit records for supported changes.

31. What security finding would you fix first?

Revoke and rotate the personal access token that remains in public repository history, then reconcile the deployed webhook and migration state. Enable leaked-password protection and enforce Premium expiry before production use.

32. What happens if Premium expires?

At the manuscript snapshot, expiry is not enforced by the client because it reads only `is_premium`. This is a known gap. Do not claim automatic re-locking until the expiry path is implemented and tested.

Testing and limitations

33. What does 902 passing tests mean?

It means the named CI release run passed its automated Flutter suite and build checks. It is strong internal evidence for covered paths, not proof that every later change, device, user group, or external service is validated.

34. What devices were actually tested?

One OPPO CPH2711 running Android 16/API 36 and desktop Chrome were audited. The declared Android minimum is API 24, but a device at that minimum, low-end phones, and tablets were not formally verified.

35. Was keyboard accessibility verified in the browser?

No. The browser audit found that keyboard traversal did not reach the demonstration controls. Do not describe the web demo as fully keyboard-operable until this is corrected and retested.

36. What did user testing show?

One remote UAT response marked all twelve games as passed, rated ease of use 5/5 and child-friendliness 4/5, and accepted the system with minor changes. Practitioner reviews produced useful revisions, including a longer Copy Me demonstration hold. Signed final practitioner validation and complete target-age parent validation remain pending.

37. Did a target-age child with an ASD diagnosis complete the assessment?

No such observed session is claimed in the manuscript. The supervised ASD participant was an adult outside the target age band, and the typically developing child sessions do not establish ASD-specific validity.

38. What is the strongest limitation of the current system?

The classifier's validity is unestablished because the data are synthetic and labels derive from the same rubric. The next strongest limitations are incomplete device/performance evidence, pending stakeholder validation, entitlement expiry, migration drift, and a shared evaluation environment.

39. What can you claim confidently?

We can claim that the prototype implements an offline-first activity and assessment loop, local persistence, a four-area classification pipeline with fallback scoring, rule-based learning paths, parent controls, synchronized records for the documented tables, server-side authorization boundaries, and an administrator portal.

40. What can you not claim yet?

We cannot claim clinical diagnosis, clinical validity, representative user acceptance, universal Android compatibility, measured frame-rate performance, keyboard accessibility of the browser demo, automatic Premium expiry enforcement, or a content-based recommender that is not implemented.

Whiteboard sequences to practice

Sequence A: assessment

```
Child plays 4 games
-> local round/session telemetry
-> 12-feature vector
-> local ONNX prediction (native runtime or browser WASM)
    unavailable -> cloud prediction, if configured
    unavailable -> deterministic rubric fallback
-> 4 skill bands + source
-> rule-based modules and starting levels
-> sequential My Path
-> parent result and history
```

Sequence B: synchronization

```
write locally
-> mark pending
-> online/authenticated?
    no: keep local and show pending state
    yes: upload in FK order
        -> upsert by UUID
        -> success: mark synced
        -> failure: keep pending, retry with backoff
```

Sequence C: payment

```
Parent taps Upgrade
-> create-checkout edge function
-> hosted PayMongo checkout
-> PayMongo signed webhook
-> HMAC verification + event deduplication
-> server writes entitlement
-> app polls its read-only entitlement row
```

Sequence D: therapy locator

```
Admin-curated centers -> cached reference list
Parent taps Find near me
-> disclosure + permission
-> one GPS fix in memory
-> Haversine ranking
-> map marker/list selection
-> external maps handoff
-> position discarded
```

Terms to define in plain language

Term	Defense definition
Offline-first	The app completes core work locally first; the network later synchronizes eligible records.
Telemetry	Structured interaction measurements collected from a gameplay session, not audio/video recording.
Feature vector	The ordered numeric input values passed to the classifier.
Multi-output classifier	One model wrapper producing a separate output for each area.
Ordinal band	An ordered educational level: Needs Support, Emerging, Strength.
Rubric fallback	Deterministic scoring used when model prediction is unavailable.
RLS	Database policies that restrict rows to the owning account or permitted role.
Idempotent upsert	Repeating an upload with the same UUID updates the same row rather than creating a duplicate.
HMAC signature	A keyed hash that lets the webhook verify that a payment event came from the expected provider.
Haversine formula	A great-circle distance calculation between two latitude/longitude points.
Data minimization	Collecting only what the stated feature needs and avoiding unnecessary media or location storage.
Parent gate	The protected transition from child mode to parent-facing functions.

Final technical defense checklist

- ☐ Memorize the 30-second system answer.
- ☐ Draw the client, local store, Supabase, edge functions, and external services.
- ☐ Explain why local-first matters in an intermittent-connectivity household.
- ☐ Name all four assessment games in order.
- ☐ Name all four prediction areas and three bands.
- ☐ Explain the 12-feature vector without calling it raw audio or diagnosis data.
- ☐ Explain the on-device -> cloud -> rubric fallback order.
- ☐ State that the current recommender is rule-based.
- ☐ Explain ordered sync, UUID upsert, pending rows, and backoff.
- ☐ Explain RLS with the two-account probe result.
- ☐ Explain why the client cannot grant Premium.
- ☐ Explain one-time in-memory GPS handling and Haversine ranking.
- ☐ Quote the 902-test result with its release scope.
- ☐ State which devices were actually tested.
- ☐ State the browser keyboard limitation.
- ☐ State the synthetic-data and label-circularity limitation.
- ☐ State the threshold discrepancy and plan to reconcile it.
- ☐ State the Premium-expiry, migration-drift, secret-history, and shared-environment gaps.
- ☐ Never describe an educational band as a diagnosis or a clinical score.

Source basis

The technical claims in this reviewer were checked against the supplied manuscript, especially the system design and architecture pages (PDF pages 64-98), security and deployment pages (PDF pages 116-131), implementation and evaluation pages (PDF pages 132-154), and the requirements/testing appendices (PDF pages 183-213 and 222-250). Repository checks included the AI assessment README and training scripts, the Dart feature aggregator, rubric scorer, recommendation rules, sync service, parent-lock service, therapy-center service, ONNX web service, game registry, administrator shell, and payment/signature functions.